



Python is Still Special

The author takes a good look at Python and discovers that he is as partial to it after years of using it, as when he first discovered it. He shares his reasons with readers.

It was in the year 2000 that I had first come across Python in the *Linux Journal*, a magazine that's no longer published. I read about it in a review titled 'Why Python' by Eric Raymond. I had loved the idea of a language that enforced indentation for obvious reasons. It was a pain to keep requesting colleagues to indent the code. IDEs were primitive then—not even as good as a simple text editor today.

However, one of Raymond's statements that stayed in my mind was, "I was generating *working* code nearly as fast as I could type."

It is hard to explain but somehow the syntax of Python offers minimal resistance!

The significance of Python even today is underlined by the fact that Uber has just open sourced its AI tool Pyro, which aims at '...deep universal probabilistic programming with Python and PyTorch (<https://eng.uber.com/pyro/>).'

Mozilla's DeepSpeech open source speech recognition module includes pre-built packages for Python (<https://goo.gl/nvXz2Y>).

Passing a function as a parameter

Years ago, after coding a number of forms, it was obvious that handling user interface forms required the same logic, except for validations. You could code a common validations routine, which used a form identifier to execute the required code. However, as the number of forms increased, it was obviously a messy solution. The ability to pass a function as a parameter in Pascal, simplified the code a lot.

So, the fact that Python can do it as well is nothing special. However, examine the simple example that follows. There should be no difficulty in reading the code and understanding its intent.

```
>>> def add(x,y):
...     return x+y
...
>>> def prod(x,y):
```

```
...     return x*y
...
>>> def op(fn,x,y):
...     return fn(x,y)
...
>>> op(add,4,5)
9
>>> op(prod,4,5)
20
>>>
```

All too often, the method required is determined by the data. For example, a form-ID is used to call an appropriate validation method. This, in turn, results in a set of conditional statements which obscure the code.

Consider the following illustration:

```
>>> def op2(fname,x,y):
...     fn = eval(fname)
...     return fn(x,y)
...
>>> op2('add',4,5)
9
>>> op2('prod',4,5)
20
>>>
```

The *eval* function allows you to convert a string into code. This eliminates the need for the conditional expressions discussed above.

Now, consider the following addition:

```
>>> newfn = """def div(x,y):
...     return x/y"""
>>> exec(newfn)
>>> div(6,2)
3
>>> op(div,6,2)
3
>>> op2('div',6,2)
```